01 / THE IDEA

Show a task. Reproduce the result.

A plain-language guide to the working video app and its proposed physical robot extension. Prepared October 8, 2026.

What the project is trying to do

A person records themselves moving objects. Software watches the video, identifies the meaningful steps, and recreates the object arrangement on screen. The proposed next stage is a robot arm performing those steps with the same objects on a real table.

One concrete example

You stand a cardboard box on its edge, then place a plastic case beside it. The app turns that demonstration into two object-placement steps. A physical robot would still need to locate each object, find a workable grip, move it, release it, and check the result.

Working today	Phone video upload, cloud video interpretation, structured placement steps, animated object playback, run history, and pipeline logs.
Proposed next	A camera and one robot arm, coordinated by the Jetson, executing a small set of tested manipulation skills.
Optional later	A learned robot policy called a VLA, trained or adapted with robot demonstrations. This is a separate development path.

What counts as copying?

The main goal is to reproduce the sequence and final relationships between objects: beside, on top, rotated, or upright. The robot can use a different movement from the person if that is what its joints and gripper can achieve.

The key distinction

The current screen output is an illustrative animation of box poses. It is not a physics-validated robot simulation, and it does not yet contain commands that can safely drive a real arm.

What the existing app does

The PC runs the application. Gemini runs in the cloud. No robot is connected to this pipeline today.

1. Phone or PC upload
QR link opens the phone upload page.

2. Local backend
Stores the video and starts a logged run.

3. Gemini: observe
Reads the video and describes object actions.

then continue below

4. Gemini: compile
Turns observed events into a structured plan.

5. Preview builder
Calculates object poses and animation frames.

6. PC dashboard
Shows video, playback, steps, and logs.

Two model calls, two different jobs

The first call identifies actions and their timing, including which object moved and what supports it. The second call converts those observations into JSON: machine-readable records containing object IDs, placement targets, and orientations.

Objects	The catalog currently contains cardboard_box and plastic_case. Their dimensions are nominal placeholders, not measurements.
Actions	PLACE and STACK represent placement steps. Repeated moves, rotation, and unstacking can be expressed through successive target poses.
Orientation	The preview accepts yaw, roll, and pitch, so an object can be shown rotated or standing on an edge.
Records	Original uploads and per-run plans, status, and event logs are stored locally. The dashboard exposes progress and recent runs.

What the animation does not prove

Its lift-transfer-lower motion is generated for visualization. It does not calculate robot joint angles or prove grip feasibility, collision avoidance, balance, or friction. A visually successful run is not yet a physical execution test.

From video to a real robot

A proposed design using the Jetson Orin Nano Developer Kit and one follower arm. A leader arm is not required for this approach.

1. Demonstration
Phone sends video to the application.

2. Cloud understanding
Gemini produces an ordered task description.

3. Live table camera
Jetson measures where the objects are now.

then continue below

4. Local task executor
Combines the goal with the measured scene.

5. Motion and arm
Tested skills produce checked joint commands.

6. Verify and report
Camera checks the outcome; dashboard logs it.

The feedback loop is the important addition

After each action, the camera observes the table again. The executor compares the actual result with the goal before continuing. For an initial system, a lost object, failed grasp, or unreachable target should stop execution and appear clearly in the logs. Bounded automatic recovery can be added later.

Cloud / Gemini	Understand the demonstration and describe intended changes. It is outside the frequent motor-control loop.
Jetson / proposed	Run camera processing, scene state, task scheduling, motion checks, and the robot driver. The backend can move here after compatibility testing.
Arm / proposed	Move its joints and gripper under local commands. It needs its own suitable power supply and a rigid mount.
PC / dashboard	Display the source video, interpreted steps, live camera, robot status, preview, and pipeline logs over the local network.

Goal first, motor commands later

A model saying "place the case beside the box" is a goal. The local system must turn that goal into a reachable gripper pose and joint motion using measured geometry and the selected arm.

Exactly where AI fits

Keep the video-understanding model separate from the software that measures and moves the robot.

Current AI model	The inspected project configuration uses gemini-3.8-flash. Both video passes use this configured Gemini model through Google's cloud API. The model ID is configuration, not a permanent architectural requirement.
VLM: in use	A vision-language model interprets visual input and produces language or structured data. Gemini serves this role here. Its placement JSON does not make it a robot-control policy.
VLA: not in use	A vision-language-action model predicts robot actions from images and instructions. The current project does not use a VLA.
Proposed perception	OpenCV ArUco marker detection and solvePnP on the Jetson. These are computer-vision algorithms, not a foundation model. Printed markers help identify and locate known objects. [3]
Proposed arm driver	If an SO-101 is selected, use LeRobot's SO101Follower interface for device connection, observations, and actions. The driver does not supply grasp planning by itself. [1, 2]
Motion planning	Custom tested skills plus inverse kinematics: mathematics that converts a desired gripper pose into joint angles. The exact solver and robot model remain to be selected for the purchased arm.
Optional future VLA	lerobot/smolvla_base is a concrete candidate: a 450-million-parameter model. It would need suitable robot data, adaptation, and evaluation. Jetson speed and memory use must be benchmarked; deployment is not established. [4]

Recommended first physical version

Use Gemini for understanding, markers for live object poses, and programmed manipulation skills for execution. This avoids making robot-policy training a prerequisite for the first working system.

What you need on the table

Already available: a phone, PC, and NVIDIA Jetson Orin Nano Developer Kit. The exact arm and camera are still decisions.

One follower arm	An SO-101 is a candidate, not a confirmed purchase. Obtain a complete arm with gripper, controller, correct power supply, USB connection, and mounting hardware. Check the actual kit contents. [1]
A fixed camera	A supported USB camera mounted above or beside the workspace. Keep the table and grasp area visible. Extra views can address occlusion later.
Suitable objects	Start with small, rigid, light objects that fit the selected gripper. The photographed boxes may be too large. Use the same objects in the demonstration and robot workspace.
Workspace tools	Rigid arm mounting, printed object markers, a calibration target, measured dimensions, and an accessible means of cutting motor power.
Jetson setup	A compatible JetPack environment, working camera and USB serial access, and appropriate power and cooling. Confirm the exact board memory and installed software. [5]

What is a leader arm?

A leader arm is a separate arm-shaped controller that a person moves by hand. Its movements can steer a follower arm and help record robot demonstrations. You do not have one, and the proposed programmed-skill approach does not require one. A follower still needs assembly and motor calibration. [1]

Why calibration matters

Camera calibration tells the software how pixels relate to geometry. Known marker sizes provide scale. A camera-to-robot transform tells the arm where those measured points are relative to its base. The marker-to-object relationship tells it where the box and a possible grasp are. All three must agree. [3]

Would LiDAR help?

Depth can help recover scale and 3D shape if usable depth data and camera calibration are available. An ordinary video file may not include that data. LiDAR is optional for a controlled, marked-object setup and does not replace robot calibration or grasp planning.

Making it work beyond the screen

The difficult part is connecting an approximate visual intention to reliable physical contact.

Physical dimensions	Approximate size is enough to illustrate an arrangement. Gripping, collision checks, and stable placement need measured geometry and appropriate mass limits.
Changing scenes	The robot must observe current object positions. It cannot assume they match the video or the preview's starting arrangement.
Hands and occlusion	Hands, the gripper, and stacked objects can hide features or markers. Use clear views, known objects, and fresh observations before each move.
Reach and orientation	A goal can be beyond the arm's reach or wrist capability. A standing box may need a different grip, an intermediate placement, or a motion the arm cannot perform.
Grasp and stability	A box can slip, deform, tip, or fall even when its target pose looks correct. Upright placement and stacking require stable support and a reliable release.
Collisions	Check the arm, gripper, objects, and table through the entire movement, not only at the destination. The current animation does not do this.
Cloud and ambiguity	Timeouts, missing evidence, or an incorrect interpretation must not become unchecked motor commands. Save the plan, validate it locally, and keep stopping independent of cloud access.

How to judge progress

Evaluate real trials: did the right object move, did it finish within defined position and orientation tolerances, did it remain stable after release, and did failures stop clearly? Record attempts and outcomes rather than treating one successful animation as evidence of physical reliability.

A practical progression

First prove repeatable physical pick-and-place. Then connect video-derived goals. Add rotation, stacking, and upright placement one at a time after verifying the arm can execute them. A future VLA is an optional extension, not a fix for missing calibration or unsuitable hardware.

What is established, what is proposed

This guide describes the inspected local application and a recommended extension. It does not claim that the physical robot system has been implemented or validated.

Current project evidence

The local codebase is video-to-robot. Relevant components are backend/server.py (uploads and dashboard), jobs.py (runs and logs), video.py (Gemini passes), free_preview.py and poses.py (object poses), preview.py (animation), data/catalog.json (known objects), and dashboard/ (interface). The inspected configuration identifies the Gemini model stated on page 4.

Decisions still needed

Choose the exact arm and compatible gripper; check that test objects fit its grasp and payload; select a supported camera; measure the workspace; and choose an inverse-kinematics implementation for that arm. The first physical actions and acceptable placement tolerances should follow these constraints.

Official technical references

[1] Hugging Face - SO-101 setup and calibration

https://huggingface.co/docs/lerobot/en/so101

[2] Hugging Face - LeRobot robot interface

https://huggingface.co/docs/lerobot/main/api/robots

[3] OpenCV - ArUco marker detection and pose estimation

https://docs.opencv.org/5.0/tutorials/objdetect/aruco_detection/aruco_detection.html

[4] Hugging Face - SmolVLA model and training

https://huggingface.co/docs/lerobot/v0.4.4/smolvla

[5] NVIDIA - Jetson Orin Nano software setup

https://docs.nvidia.com/jetson/orin-nano-devkit/user-guide/latest/setup_bsp.html

Plain-language glossary

Pose: an object's position and orientation. JSON: structured data software can read. Inverse kinematics: calculating joint angles for a gripper target. Feedback: measuring what happened and using it to decide the next step. LeRobot: robotics software and tooling; it is not the name of one specific arm.